

Sistema Web de gestión

Gimnasio STRKE

Presentado por

Grupo 02

Día

02 de Junio de 2026

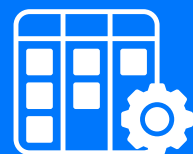
Repositorio

[gym-management-system](#)

Curso:

Herramientas de desarrollo

Herramienta de Gestión: Jira



¿Qué es Jira?

Plataforma de gestión de proyectos de Atlassian que permite planificar, rastrear y organizar el trabajo del equipo mediante tickets, sprints y tableros visuales.



¿Cómo funciona?

Permite registrar tareas, asignar responsables, establecer prioridades y hacer seguimiento del avance actualizando el estado de cada actividad: Por hacer → En progreso → Listo.



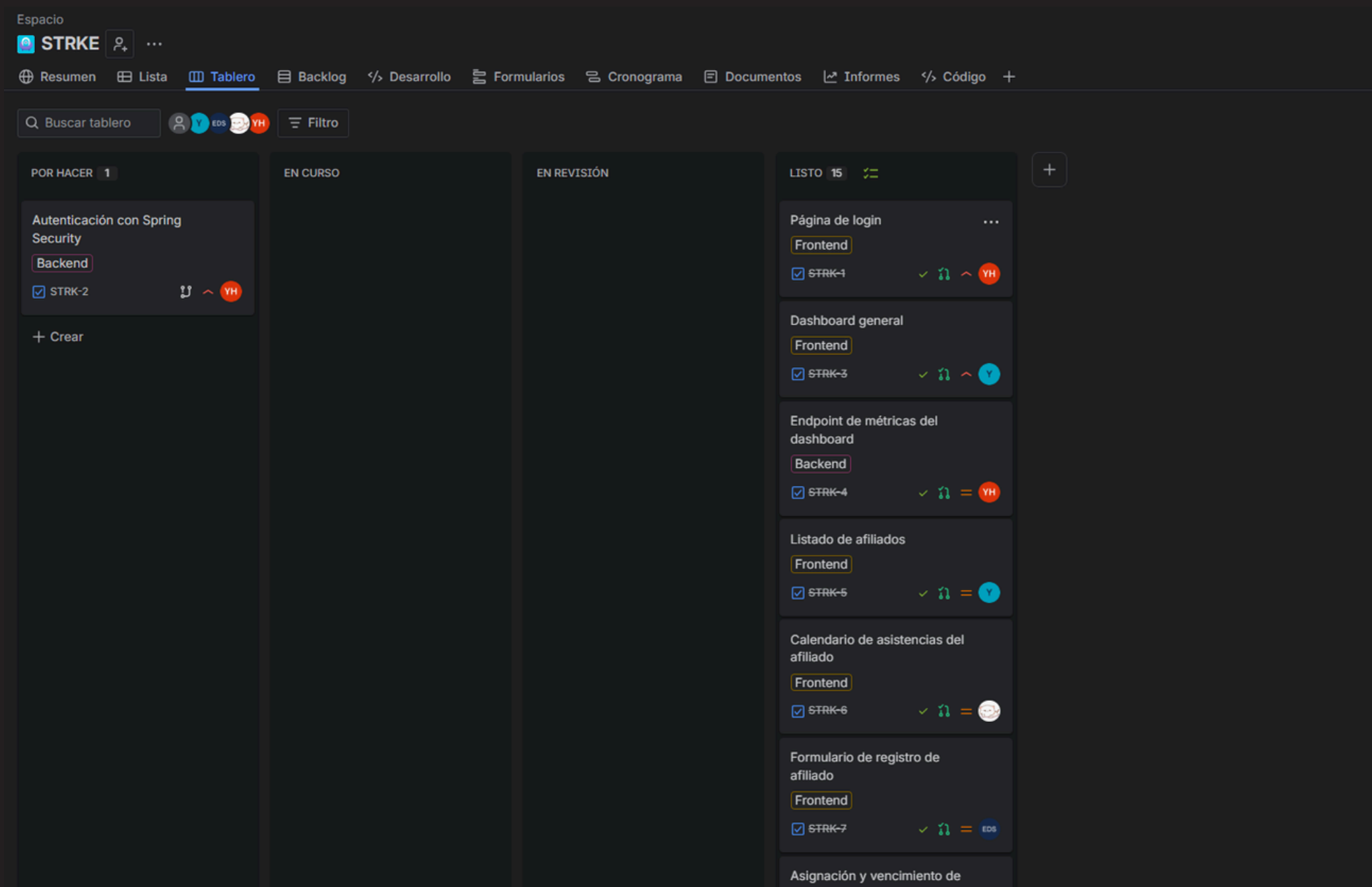
Beneficios

Optimiza la gestión del trabajo al centralizar tareas, responsables y estados en un único entorno, garantizando trazabilidad y control durante todo el desarrollo.

Sprint 1 - 16 actividades

☐ STRK Sprint 1 29 may – 2 jun (16 actividades)	000	Completar sprint	...
Implementar módulos de autenticación, dashboard y afiliados			
☒ STRK-1 Página de login	LISTO	- ^ YH	
☒ STRK-2 Autenticación con Spring Security	POR HACER	- ^ YH	
☒ STRK-3 Dashboard general	LISTO	- ^ Y	
☒ STRK-4 Endpoint de métricas del dashboard	LISTO	- = YH	
☒ STRK-5 Listado de afiliados	LISTO	- = Y	
☒ STRK-6 Calendario de asistencias del afiliado	LISTO	- =	
☒ STRK-7 Formulario de registro de afiliado	LISTO	- = EDS	
☒ STRK-15 Asignación y vencimiento de membresías	LISTO	- = EDS	
☒ STRK-11 Registro de asistencia	LISTO	- = EDS	
☒ STRK-8 CRUD de afiliados	LISTO	- ^ YH	
☒ STRK-9 Búsqueda de afiliados por DNI/nombre	LISTO	- = YH	
☒ STRK-10 Control de asistencia	LISTO	- =	
☒ STRK-12 Asignación de membresía	LISTO	- = Y	
☒ STRK-13 Configuración de membresías	LISTO	- =	
☒ STRK-14 CRUD de tipos de membresía	LISTO	- =	
☒ STRK-16 Configurar Bootstrap en proyecto Spring Boot con Thymeleaf	LISTO	- = YH	

Tablero Kanban

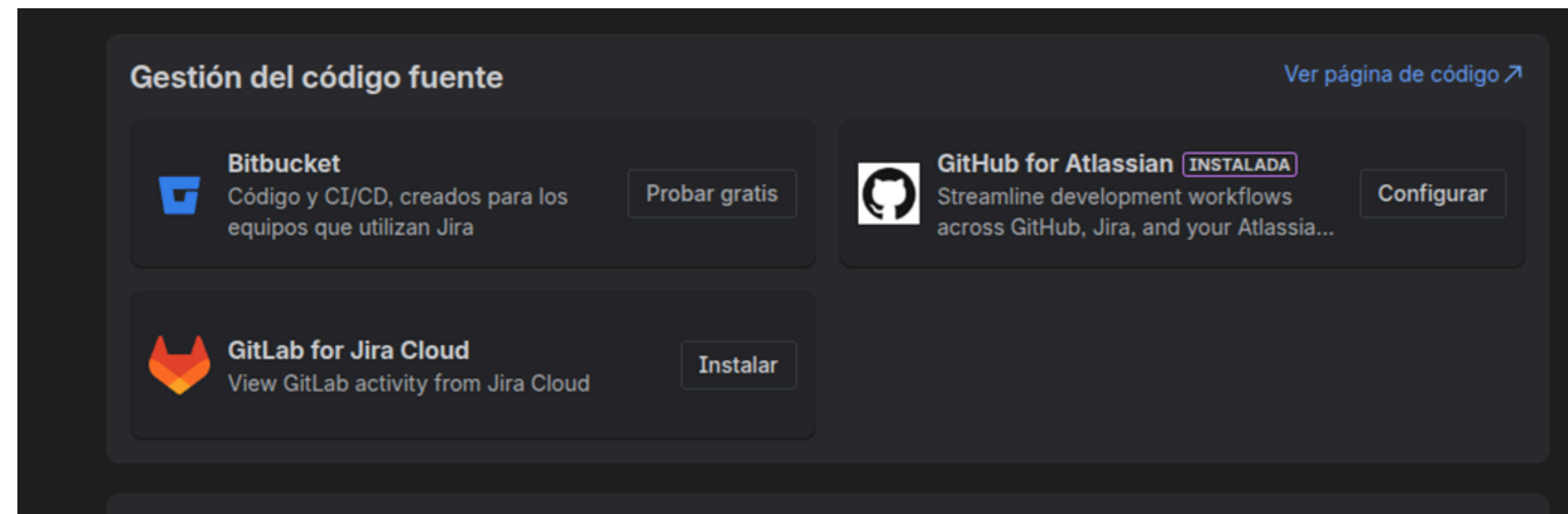


El tablero **Kanban** muestra las tareas organizadas en columnas según su estado: Por hacer, En curso, En revisión y Listo. Permite visualizar el avance del sprint en tiempo real, identificando rápidamente qué tareas están pendientes y cuáles ya fueron completadas.

Integración entre GitHub y Jira

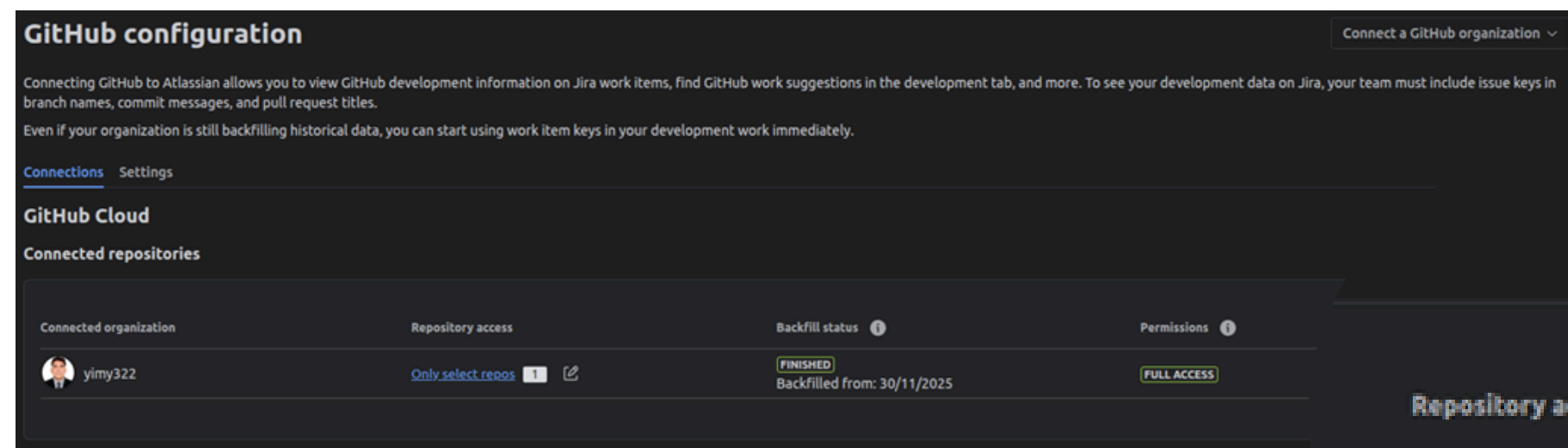
Paso 1

Instalar el plugin de GitHub



Paso 2

Conectar con una cuenta de github y **seleccionar** los repositorios que queremos vincular



Repository access

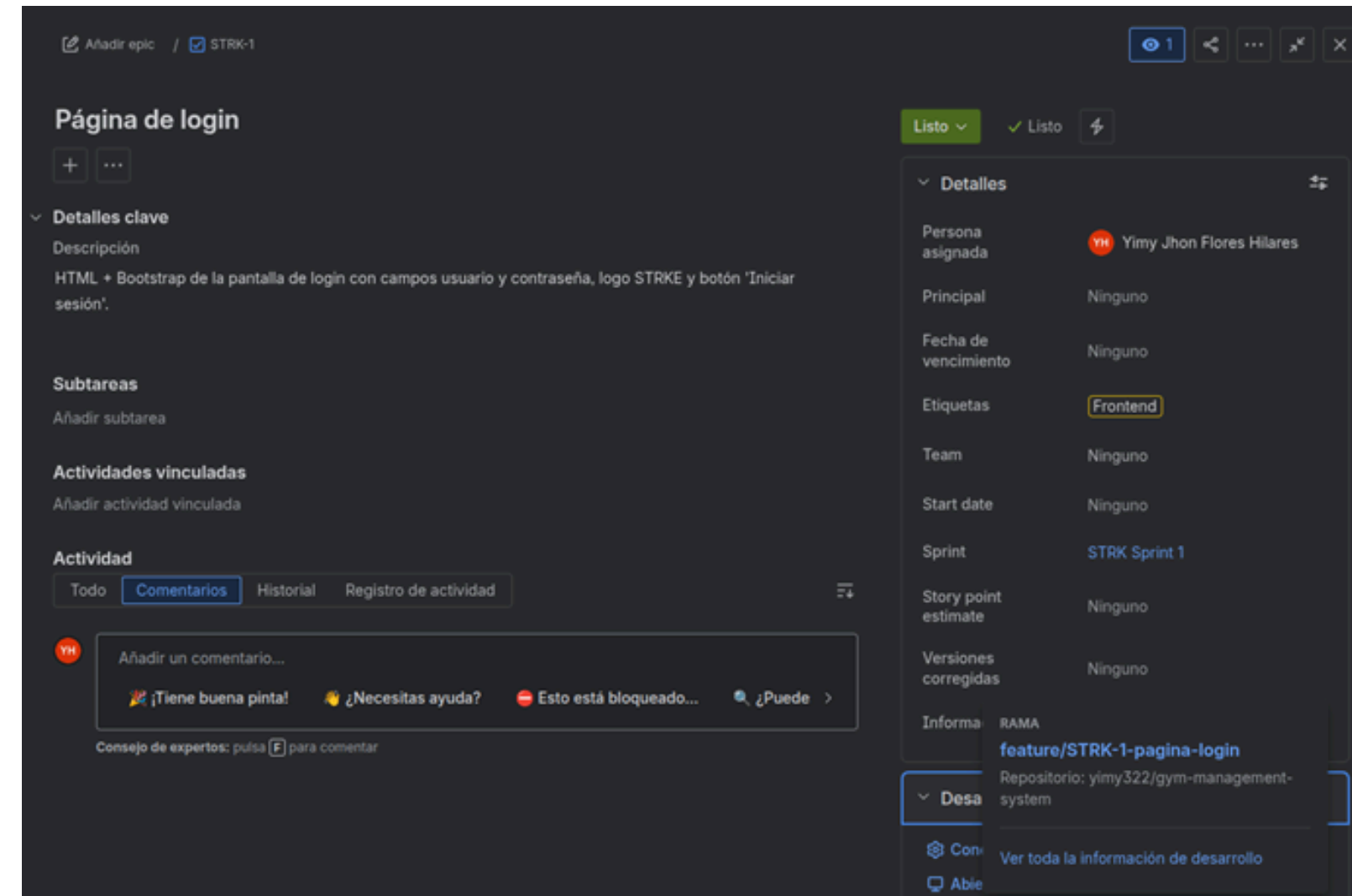
[Only select repos](#) 1 

Integración entre GitHub y Jira

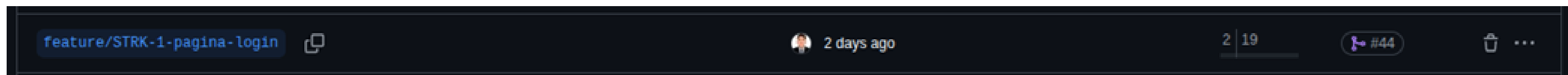
Paso 3

Usamos el identificador del issue (ej. STRK-1) en el nombre de las ramas y commits para vincular cada cambio de código con su tarea correspondiente en Jira.

*Esto nos ayuda a mantener la trazabilidad



Rama: feature/STRK-1-pagina-login vista desde el github



Integración entre GitHub y Jira

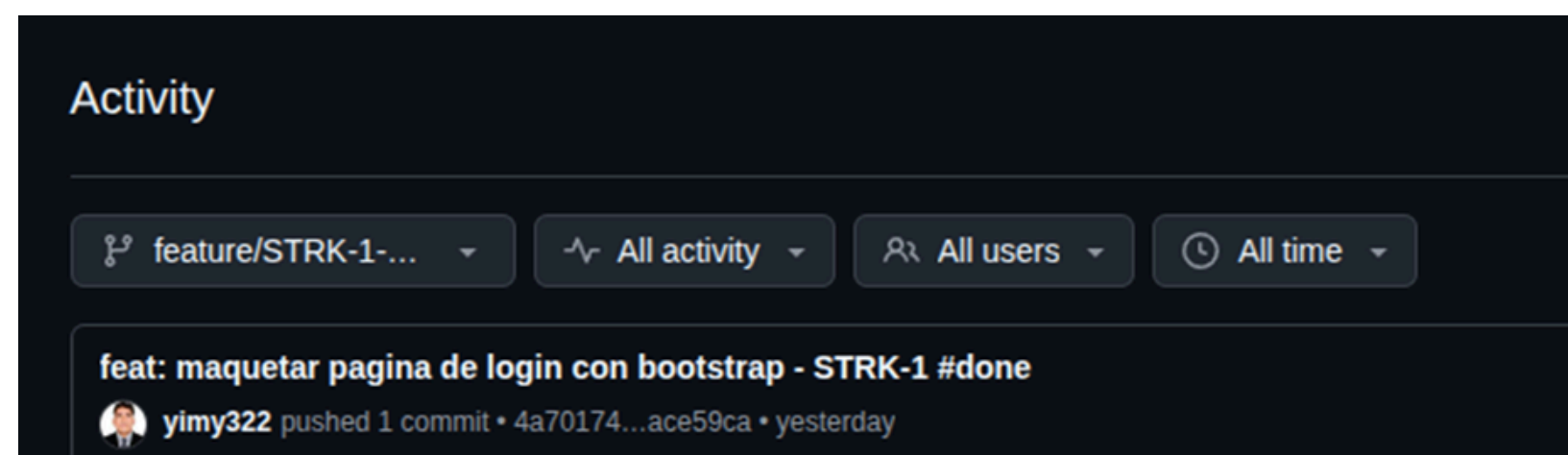


Resultado

Rama: feature/STRK-1-pagina-login **vinculada** en el issue correspondiente gracias al identificador **STRK-1**

Resultado

Commit: “feat: maquetar pagina de login con bootstrap - STRK-1 #done”

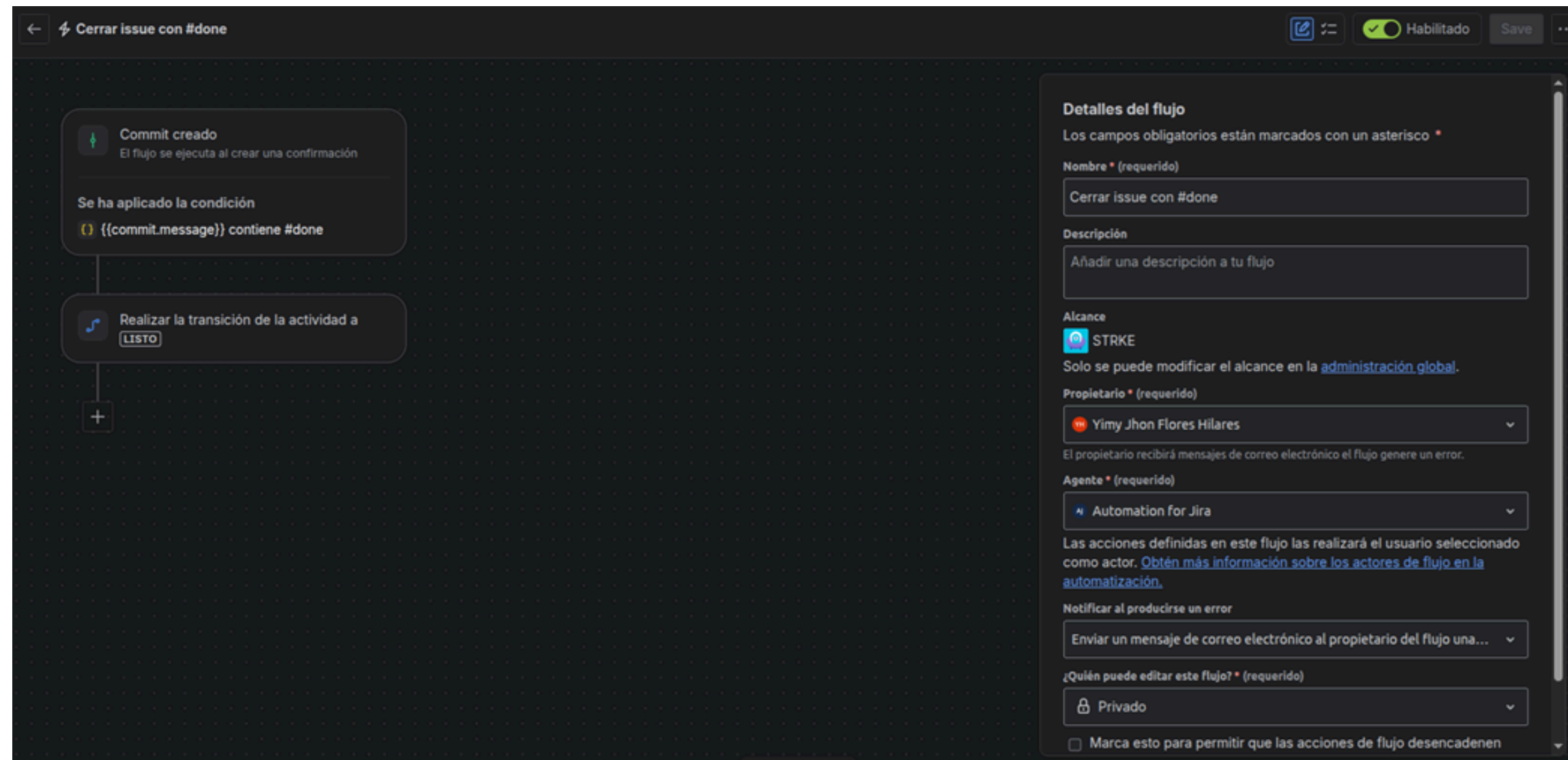


El mensaje del commit incluye el identificador del issue (STRK-1) para vincularlo con Jira y #done para cerrarlo automáticamente.

Integración entre GitHub y Jira

Paso 4

Configuramos una regla de automatización en Jira para cerrar el issue automáticamente cuando un **commit** incluye **#done** en su mensaje.



The screenshot shows the Jira Automation interface for a rule titled "Cerrar issue con #done". The rule is configured with the following steps:

- Commit creado**: El flujo se ejecuta al crear una confirmación.
- Se ha aplicado la condición**: `{{commit.message}}` contiene #done.
- Realizar la transición de la actividad a**: LISTO.

The right sidebar shows the "Detalles del flujo" (Flow Details) section, which includes fields for:

- Nombre *** (required): Cerrar issue con #done.
- Descripción**: Añadir una descripción a tu flujo.
- Alcance**: STRKE. Solo se puede modificar el alcance en la [administración global](#).
- Propietario *** (required): Yimy Jhon Flores Hilares.
- Agente *** (required): Automation for Jira.
- Notificar al producirse un error**: Enviar un mensaje de correo electrónico al propietario del flujo una...
- ¿Quién puede editar este flujo? *** (required): Privado.
- ☐ Marca esto para permitir que las acciones de flujo desencadenen...

Resolución de Merge Conflict

Paso 1

Al fusionar la rama **develop** con **feature/STRK-2-spring-security** se generó un conflicto en los archivos modificados por ambas ramas simultáneamente.

```
yimy@PC-322: ~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git checkout feature/STRK-2-spring-security
Switched to branch 'feature/STRK-2-spring-security'
Your branch is up to date with 'origin/feature/STRK-2-spring-security'.
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git merge develop
Auto-merging src/main/resources/templates/fragments/navbar.html
CONFLICT (content): Merge conflict in src/main/resources/templates/fragments/navbar.html
Automatic merge failed; fix conflicts and then commit the result.
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

Paso 2

Identificamos el **conflicto** en el archivo, donde VS Code marcó los cambios de ambas ramas: **HEAD** con el botón de logout con formulario y **develop** con un enlace simple.

```
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
<<<<<<< HEAD (Current Change)
|
|   <form th:action="@{/logout}" method="post">
|       <input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}"/>
|       <button type="submit" class="btn btn-sm btn-outline-secondary rounded-circle p-1" style="width:32px; height:32px;">
|           <i class="fa-solid fa-arrow-right-from-bracket" style="font-size:12px;"></i>
|       </button>
|   </form>
|
=====
|   <a class="btn btn-danger" style="width:32px; height:32px;">
|       <i class="fa-solid fa-arrow-right-from-bracket" style="font-size:12px;"></i>
|   </a>
>>>>>> develop (Incoming Change)
|
|   </div>
```

Resolución de Merge Conflict

Paso 3

Conservamos el cambio de nuestra rama actual, manteniendo el botón de **logout** con su formulario de **seguridad**.

```
<form th:action="@{/logout}" method="post">
  <input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}"/>
  <button type="submit" class="btn btn-sm btn-outline-secondary rounded-circle p-1" style="width:32px; height:32px;">
    <i class="fa-solid fa-arrow-right-from-bracket" style="font-size:12px;"></i>
  </button>
</form>
</div>
```

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git add .
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git commit -m "fix: resolucion conflicto navbar logout"
[feature/STRK-2-spring-security 065c484] fix: resolucion conflicto navbar logout
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git push origin feature/STRK-2-spring-security
Username for 'https://github.com': yimy322
Password for 'https://yimy322@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 227 bytes | 227.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/yimy322/gym-management-system.git
 9a6a2d8..065c484 feature/STRK-2-spring-security -> feature/STRK-2-spring-security
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

git add .

git commit -m "fix: resolucion conflicto navbar logout"

git push origin feature/STRK-2-spring-security

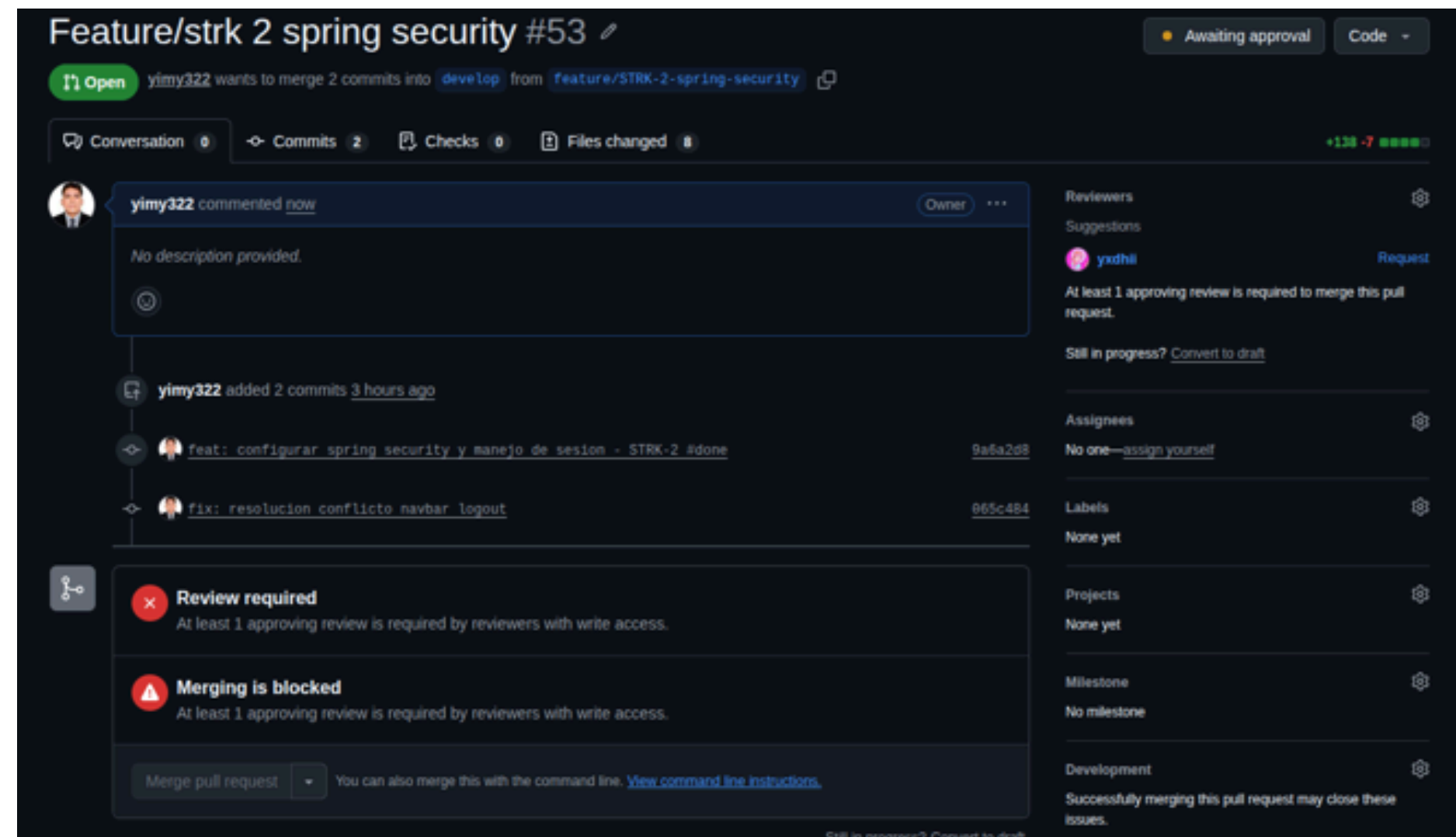
Paso 4

Agregamos el archivo corregido, realizamos un **commit** indicando la resolución del conflicto y subimos el **cambio** a la rama remota.

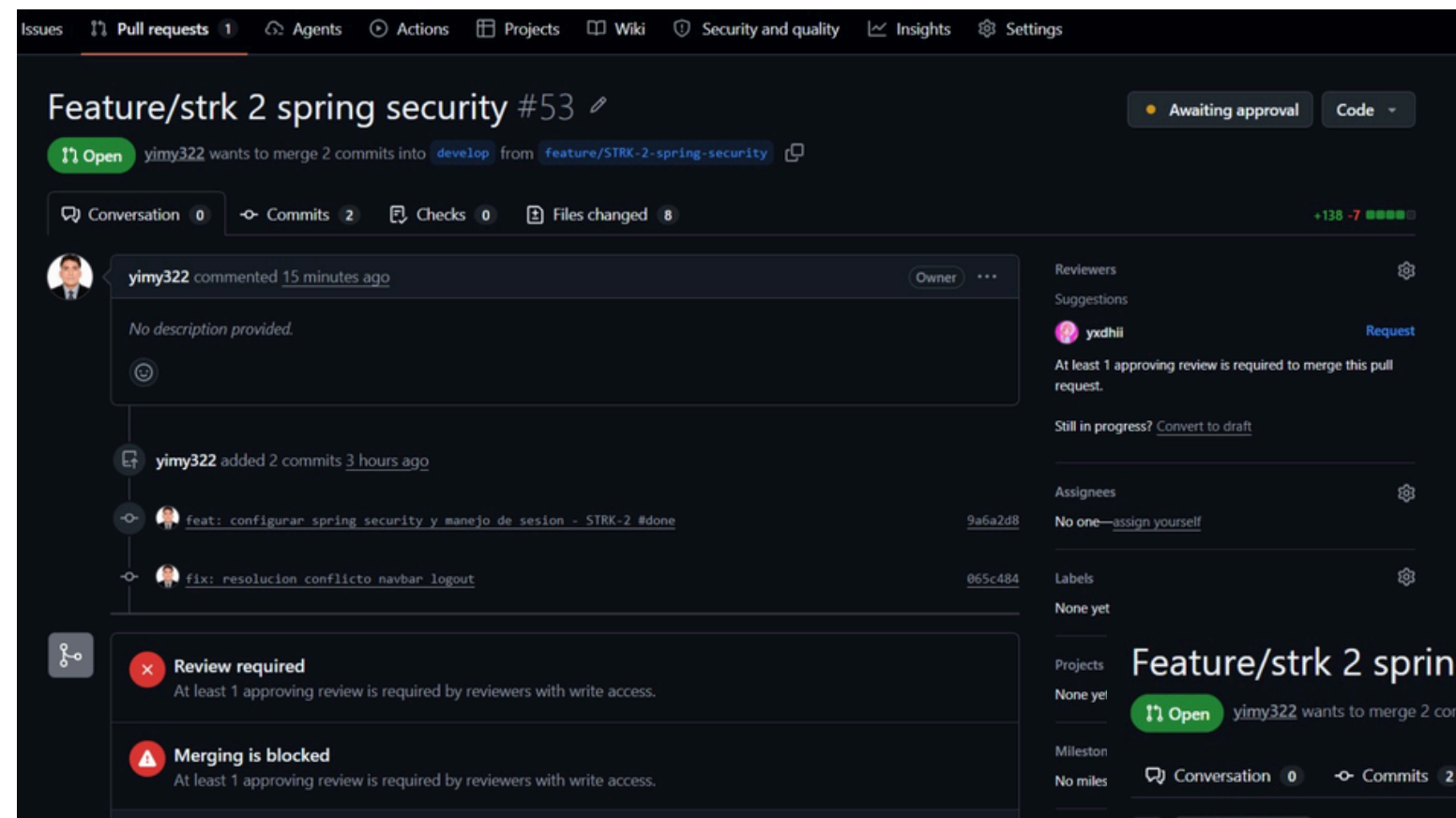
Resolución de Merge Conflict

Paso 5

Con el conflicto resuelto, se creó el **Pull Request** de **feature/STRK-2-spring-security** hacia **develop**, quedando a la espera de aprobación para su fusión.



Revisión y aprobación del Pull Request

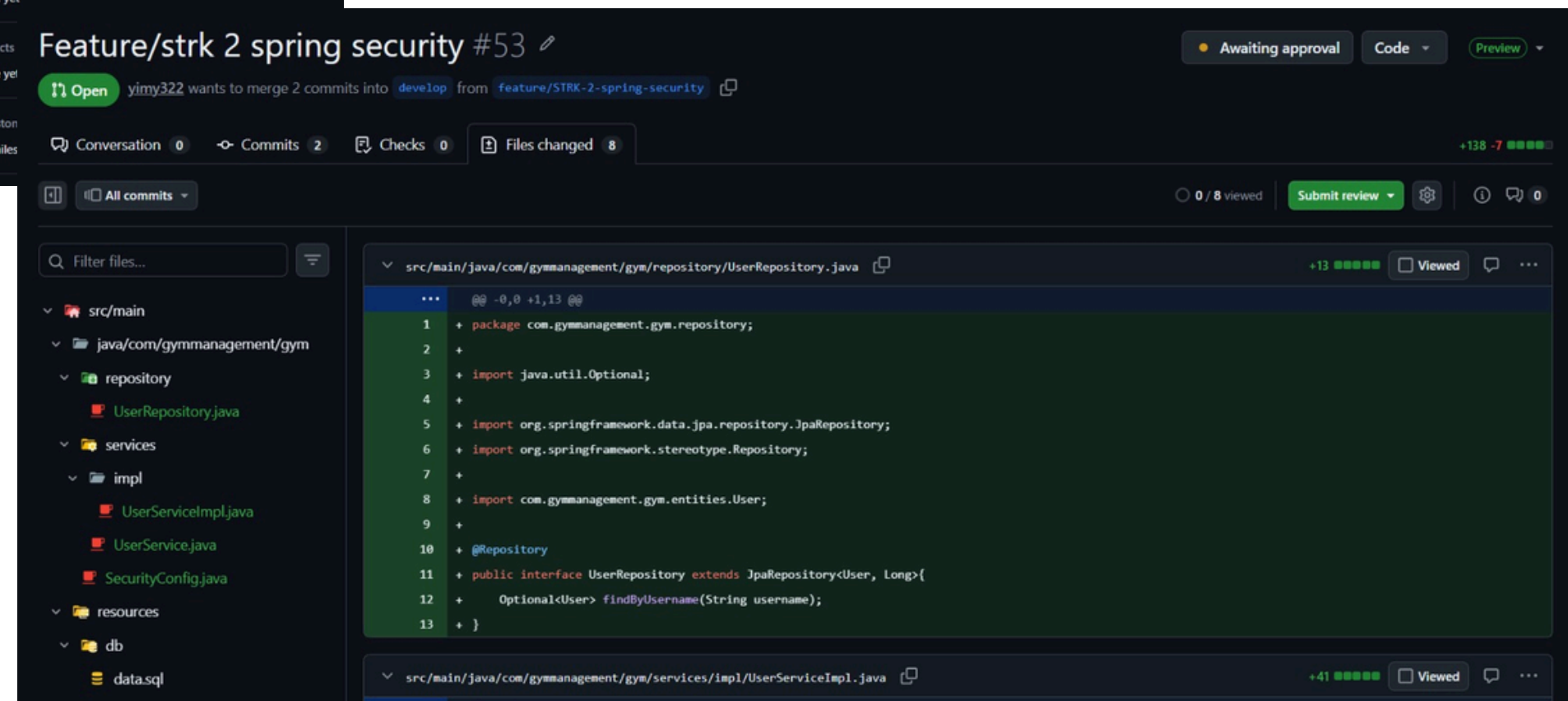


Paso 1

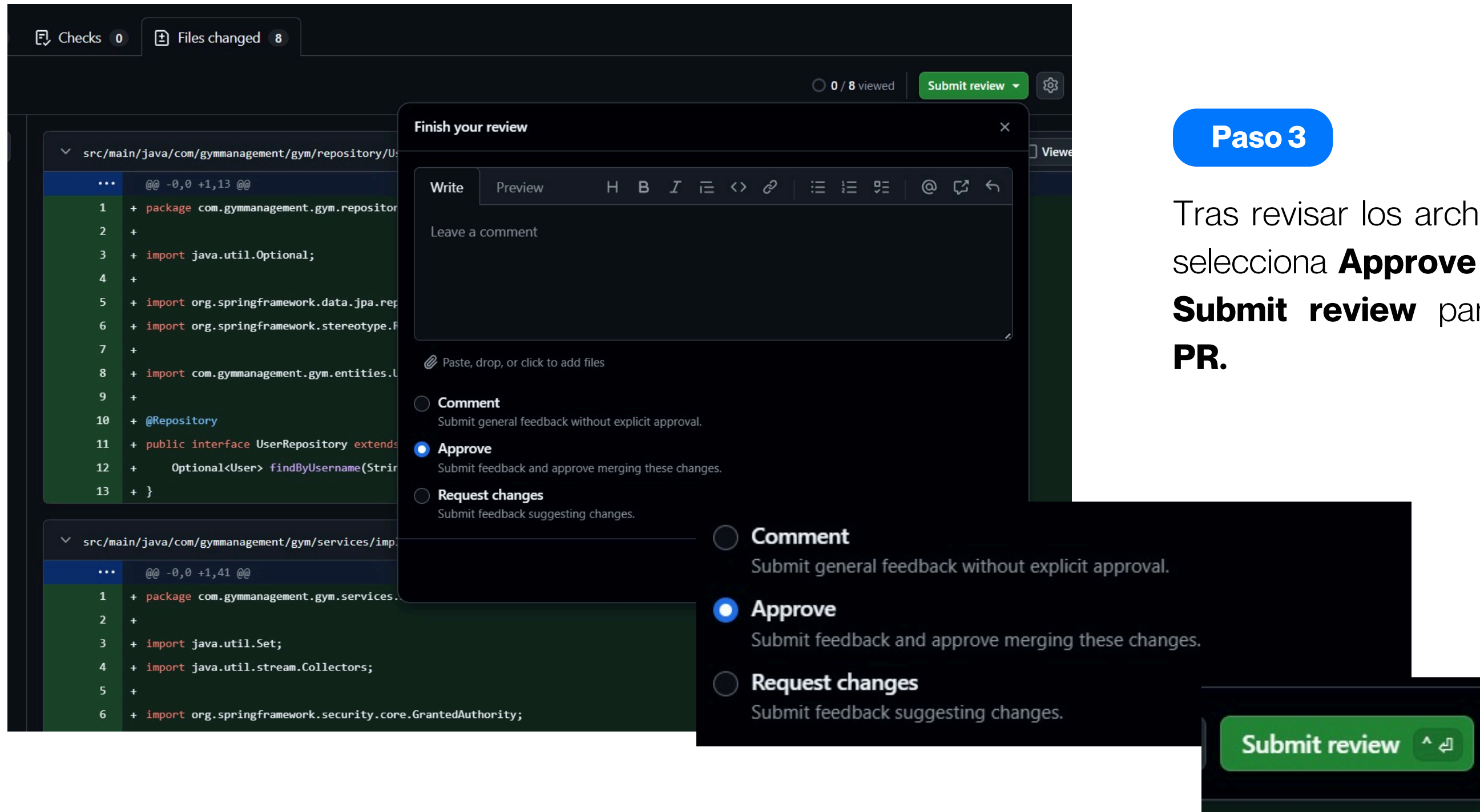
Se abre el **Pull Request feature/STRK-2-spring-security** hacia **develop** con 2 commits, quedando en espera de aprobación del revisor (**yxdhii**)

Paso 2

El revisor accede a la pestaña **Files changed** para revisar los 8 archivos modificados **antes de aprobar el PR**.



Revisión y aprobación del Pull Request

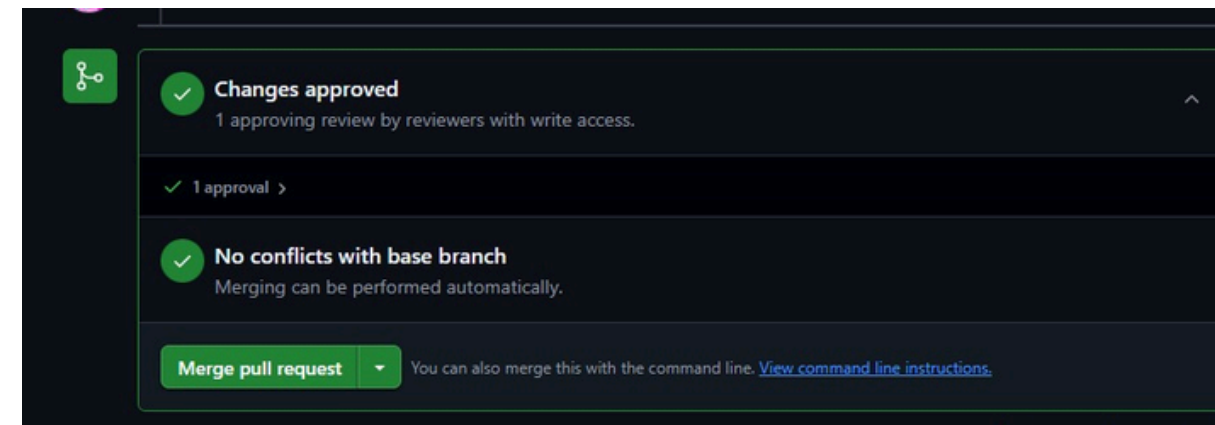
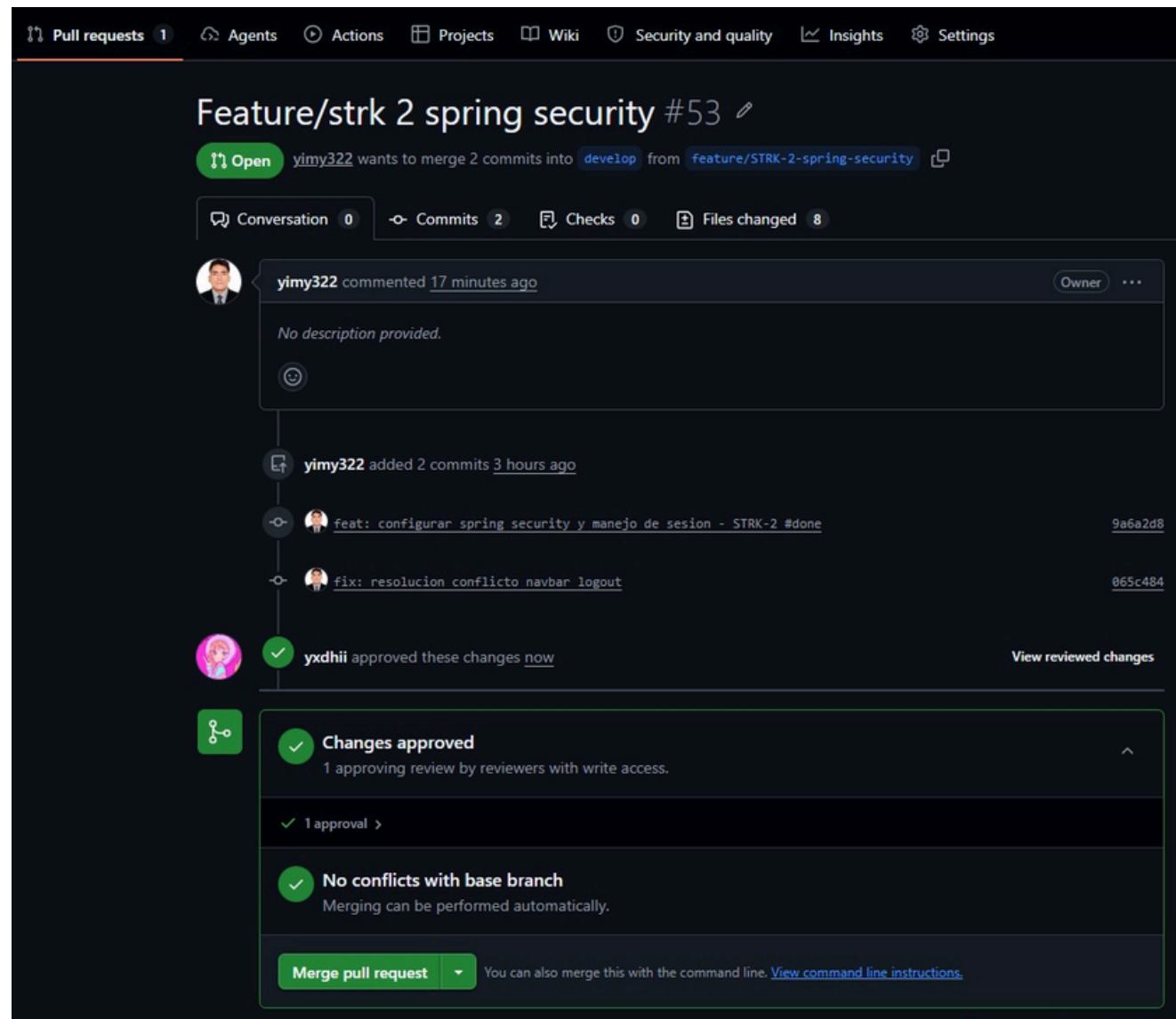


The screenshot displays a GitHub Pull Request review interface. At the top, there are tabs for 'Checks' (0) and 'Files changed' (8). A 'Submit review' button is visible in the top right. The main area shows a diff of code changes. On the left, the file path is 'src/main/java/com/gymmanagement/gym/repository/UserRepository.java'. The diff shows changes to the package, imports, and the 'findByUsername' method. On the right, the 'Finish your review' modal is open, showing a 'Write' tab with a text area for comments. Below the text area, there are three radio button options: 'Comment' (Submit general feedback without explicit approval), 'Approve' (Submit feedback and approve merging these changes), and 'Request changes' (Submit feedback suggesting changes). The 'Approve' option is selected. At the bottom right, there is a 'Submit review' button with a dropdown arrow.

Paso 3

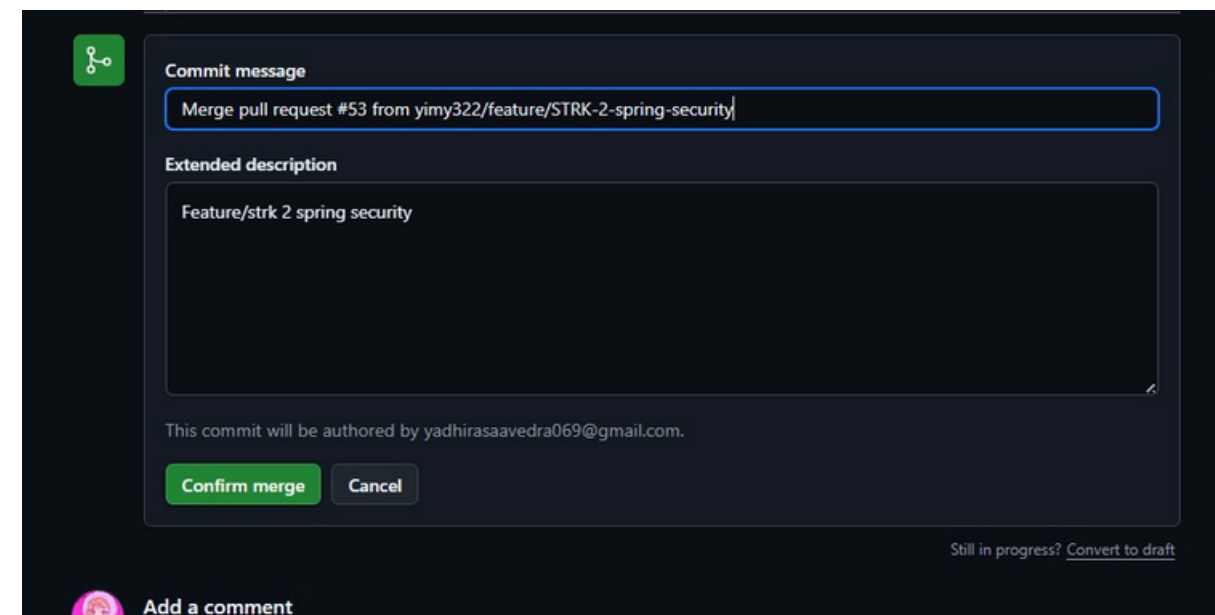
Tras revisar los archivos, el revisor selecciona **Approve** y hace clic en **Submit review** para aprobar el PR.

Revisión y aprobación del Pull Request



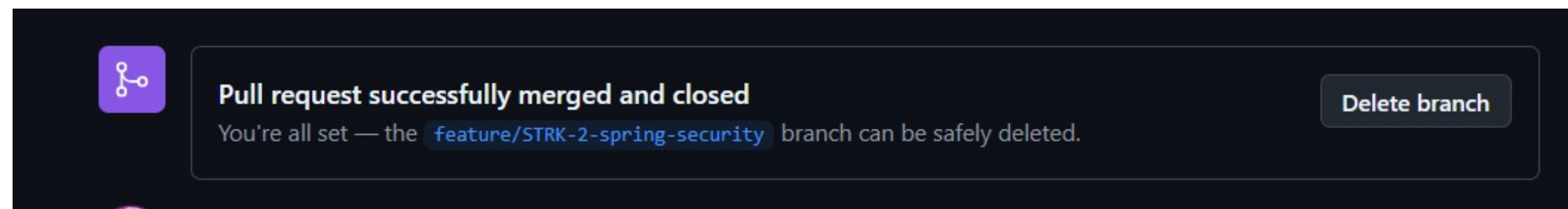
Paso 4

Una vez aprobado, el PR muestra **Changes approved** y **No conflicts with base branch**, habilitando el botón para realizar el **Merge pull request**.



Paso 5

Se confirma el merge con **Confirm merge**, fusionando **feature/STRK-2-spring-security** en **develop** exitosamente y cerrando el **PR**.



Creación de Release y Tag

Paso 1

Nos ubicamos en la rama **develop** ejecutando **git checkout develop**.

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

Paso 2

Se crea la rama del release ejecutando **git checkout -b release/v2.0-entrega-parcial-2**

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git checkout -b release/v2.0-entrega-parcial-2
Switched to a new branch 'release/v2.0-entrega-parcial-2'
```

Paso 3

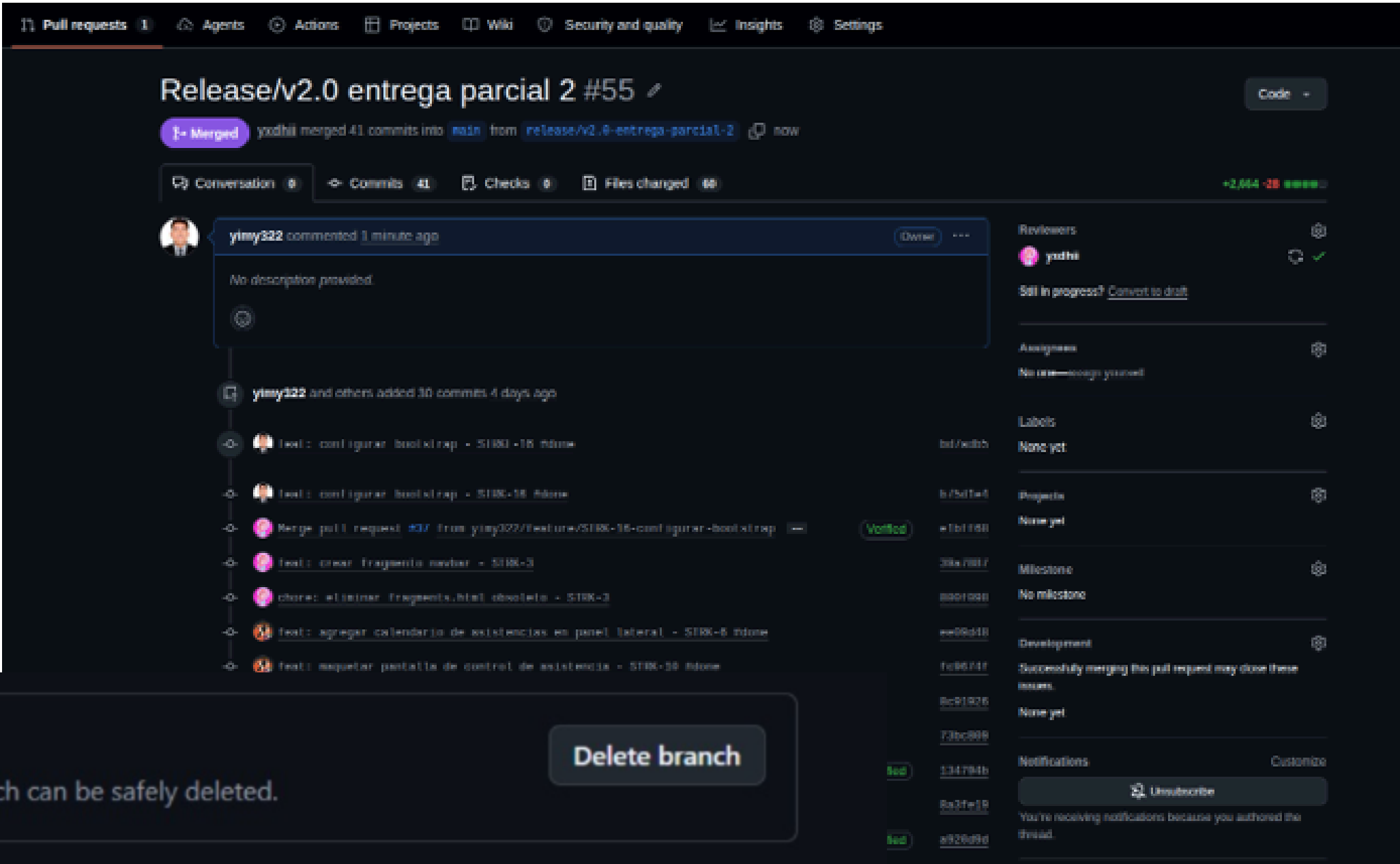
Se sube la rama release al repositorio remoto ejecutando **git push -u origin release/v2.0-entrega-parcial-2**.

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git push -u origin release/v2.0-entrega-parcial-2
Username for 'https://github.com': yimy322
Password for 'https://yimy322@github.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'release/v2.0-entrega-parcial-2' on GitHub by visiting:
remote:   https://github.com/yimy322/gym-management-system/pull/new/release/v2.0-entrega-parcial-2
remote:
To https://github.com/yimy322/gym-management-system.git
 * [new branch]      release/v2.0-entrega-parcial-2 -> release/v2.0-entrega-parcial-2
Branch 'release/v2.0-entrega-parcial-2' set up to track remote branch 'release/v2.0-entrega-parcial-2' from 'origin'.
```


Creación de Release y Tag

Paso 4

Se crea el **Pull Request** de **release/v2.0-entrega-parcial-2** hacia **main**, consolidando los **41 commits** del proyecto. El revisor aprueba y se realiza el **merge** exitosamente.



The screenshot shows a GitHub Pull Request interface for "Release/v2.0 entrega parcial 2 #55". The status is "Merged" and it shows "yodhi merged 41 commits into main from release/v2.0-entrega-parcial-2". The commit history includes several feature branches like "feat: configurar bootstrap" and "feat: crear fragmento navbar". A success message at the bottom states: "Pull request successfully merged and closed. You're all set — the release/v2.0-entrega-parcial-2 branch can be safely deleted." A "Delete branch" button is also visible.

Creación de Release y Tag

Paso 5

Con el merge aprobado, se crea el **tag** en **main** y se sube al repositorio remoto.

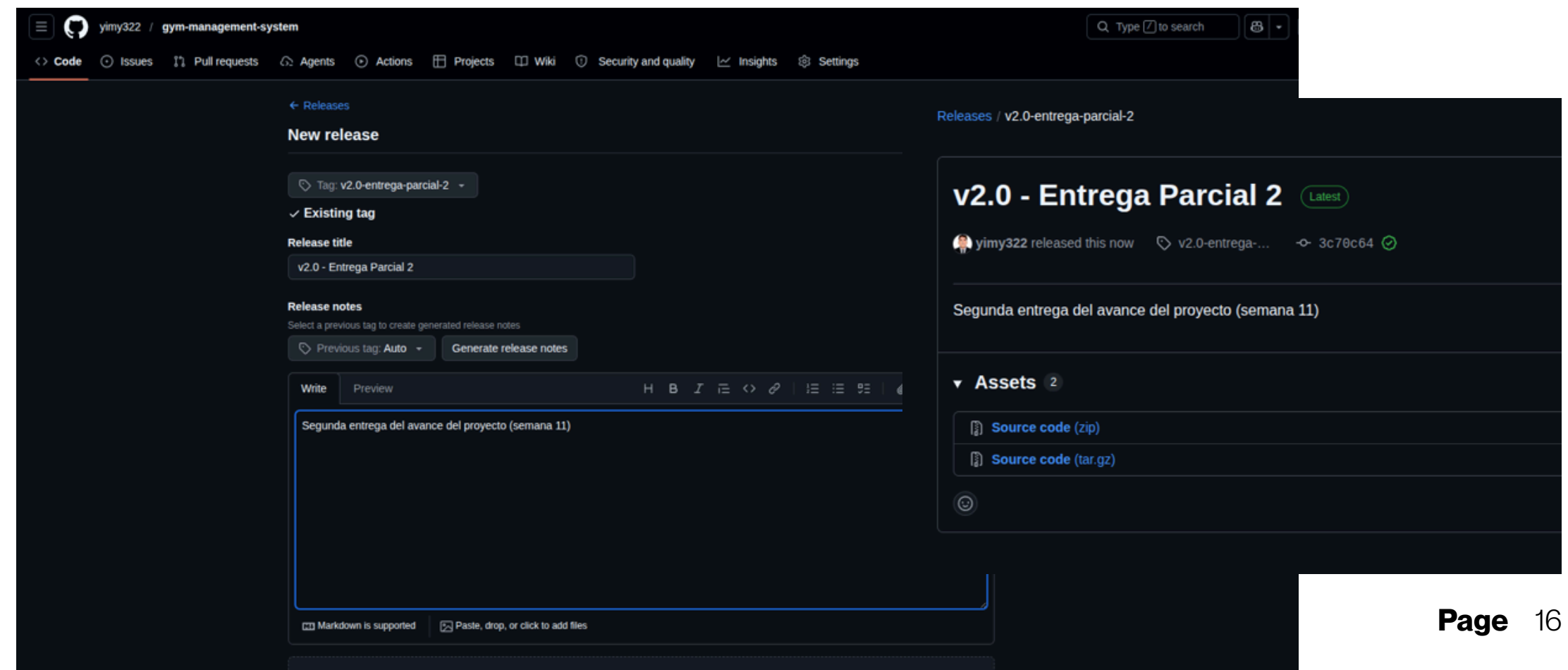
```
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-managment-system$ git tag -a v2.0-entrega-parcial-2 -m "Avance 2 completo"
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-managment-system$ git push origin v2.0-entrega-parcial-2
Username for 'https://github.com': yiny322
Password for 'https://yiny322@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 172 bytes | 172.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/yiny322/gym-managment-system.git
 * [new tag]          v2.0-entrega-parcial-2 -> v2.0-entrega-parcial-2
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-managment-system$
```

`git tag -a v2.0-entrega-parcial-2 -m "Avance 2 completo"`

`git push origin v2.0-entrega-parcial-2`

Paso 6

En **GitHub** se crea el release seleccionando el **tag v2.0-entrega-parcial-2**, se agrega el título "**v2.0 - Entrega Parcial 2**" y la descripción "**Segunda entrega del avance del proyecto (semana 11)**", generando automáticamente el código fuente descargable en .zip y .tar.gz



Herramienta de Colaboración: Trello



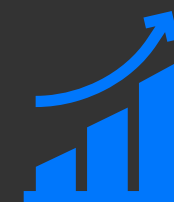
¿Qué es Trello?

Herramienta visual de gestión de tareas basada en tableros Kanban que permite organizar actividades mediante tarjetas distribuidas en columnas según su estado.



¿Cómo funciona?

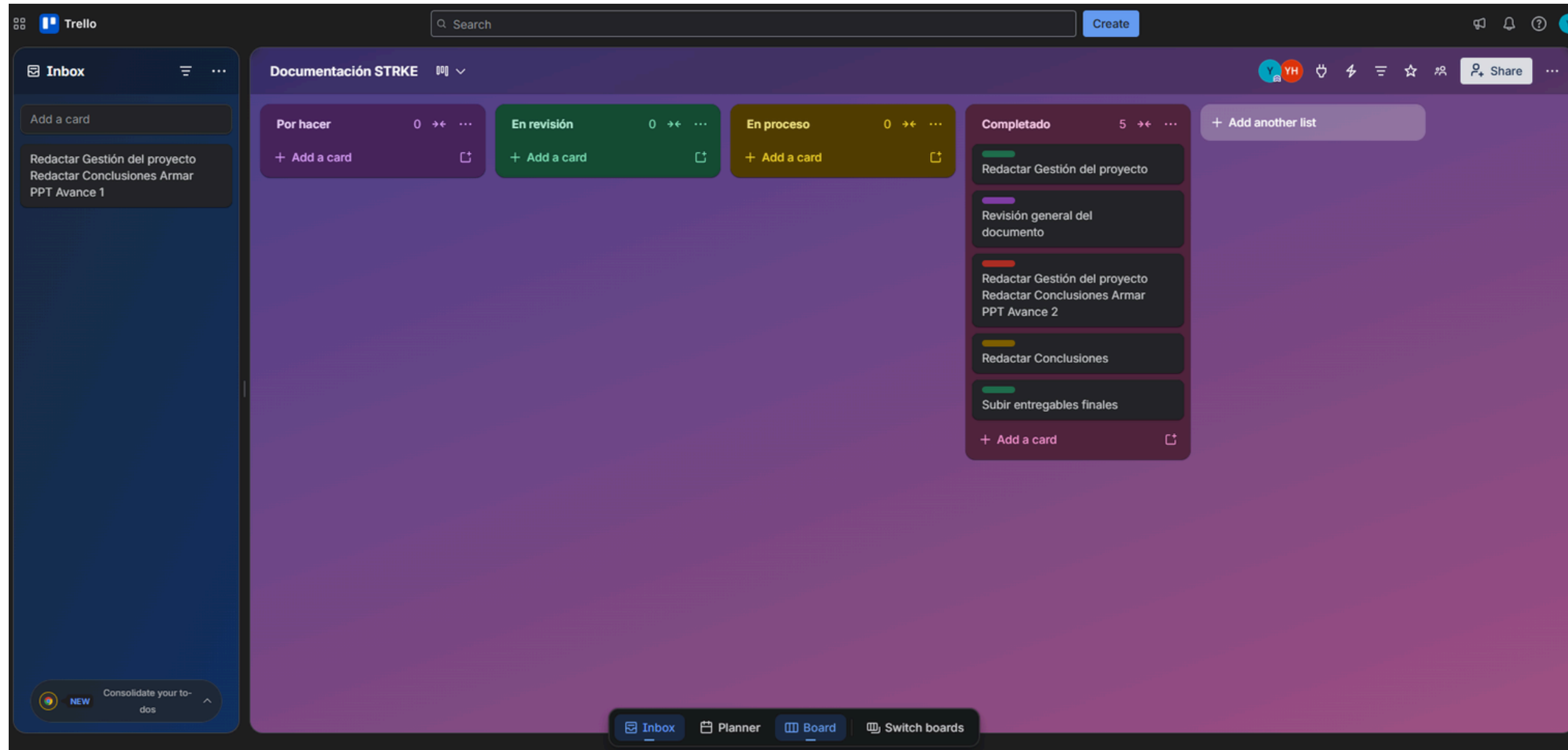
El equipo crea tarjetas por cada actividad, las organiza en columnas según su estado y actualiza su avance moviéndolas de Por hacer a Completado.



Beneficios

Facilita la organización de actividades, mejora la visibilidad del avance del equipo y centraliza la gestión de tareas de documentación en un solo tablero.

Tablero de Documentación — Trello



Se utilizó **Trello** para organizar y dar seguimiento a las actividades de documentación del proyecto, distribuyendo las tareas en columnas según su estado hasta completar los 5 entregables.

Conclusiones

01.

Trabajo colaborativo

La integración entre GitHub y Jira fortaleció el trabajo en equipo, vinculando cada rama y commit con su issue correspondiente y automatizando el cierre de tareas mediante #done.

02.

Organización del proyecto

El uso de sprints y el tablero Kanban en Jira permitió distribuir las actividades de forma equitativa y mantener visibilidad del avance en tiempo real durante el desarrollo del sistema.

03.

Gestión de tareas

La resolución del Merge Conflict, la revisión de Pull Requests y el uso de Trello para la documentación demostraron la importancia de establecer flujos de trabajo claros y colaborativos.

MUCHAS GRACIAS

por su atención.